Graphical Galois Slicing

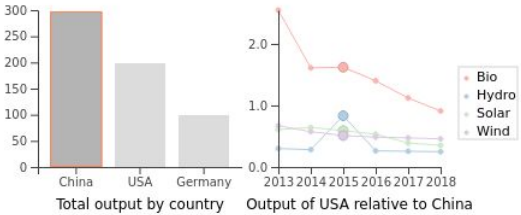
Joseph Bond, James Cheney, Cristina David, Minh Nguyen and Roly Perera

Contents

Background and Problem Statement

Background: Fluid

- Scientists often need to interpret computational models
- Models communicate information through charts and figures
  - Understanding charts/figures means understanding what visual elements *represent*, how they were computed and the data they used
  - Even for an expert can be difficult just given the source-code and dataset
- Fluid is a project that Minh, Roly and myself have been working on, aiming to make computational outputs transparent and self-explanatory[1]



year	country	energyType	output
2015	China	Bio	10.3
2015	China	Hydro	96
2015	China	Solar	44
2015	China	Wind	145

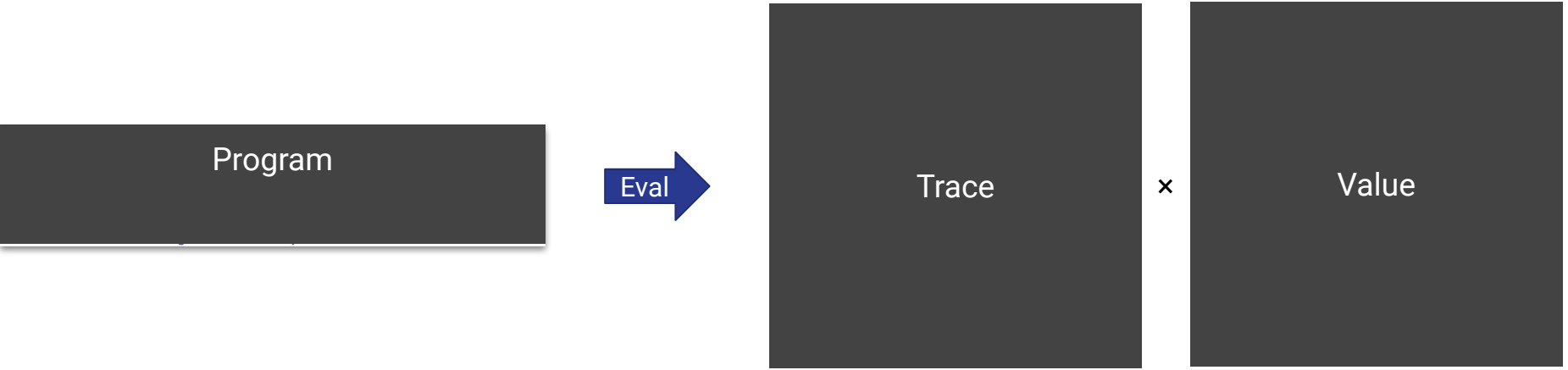
(1) Roly Perera, Minh Nguyen, Tomas Petricek, and Meng Wang. 2022. Linked visualisations via Galois dependencies. Proc. ACM Program. Lang. 6, POPL, Article 7 (January 2022), 29 pages. https://doi.org/10.1145/3498668

Fluid

```
let filter' = [[0, 0, 0],
               [3, 7, 1],
               [0, 0, 0]];
    filter  = [| nth2 i j filter' | (i, j) in (3, 3) |];
    image   = [| nth2 i j data | (i, j) in (5, 5) |]
in convolve image filter wrap
```

- Call-by-value dynamically typed functional calculus
  - Surface and core syntax
  - Records, algebraic datatypes and pattern-matching
- Bi-directional dependency Analysis
  - Make selections on output and compute the necessary input
  - Selection information can be followed through both evaluation and desugaring stage

System Overview



System Overview: Evaluation

```
1  let filter' = [[0, 0, 0],
2                 [3, 7, 1],
3                 [0, 0, 0]];
4      filter  = [| nth2 i j filter' | (i, j) in (3, 3) |];
5      image   = [| nth2 i j data | (i, j) in (5, 5) |]
6  in convolve image filter wrap
```

Eval

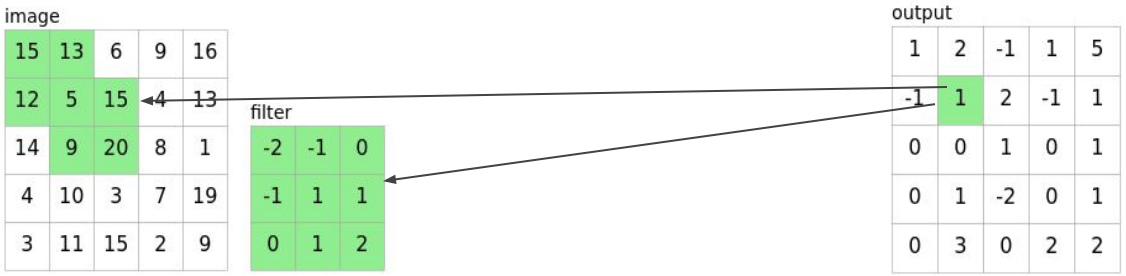
Trace

×

output

1	2	-1	1	5
-1	1	2	-1	1
0	0	1	0	1
0	1	-2	0	1
0	3	0	2	2

Backwards Slicing




```
1  let filter' = [[0, 0, 0],
2                 [3, 7, 1],
3                 [0, 0, 0]];
4      filter  = [| nth2 i j filter' | (i, j) in (3, 3) |];
5      image  = [| nth2 i j data | (i, j) in (5, 5) |]
6  in convolve image filter wrap
```

How Does it Work?

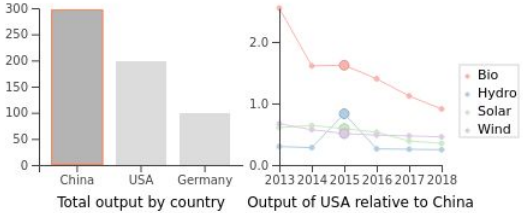
- Don’t just compute a value as the result of a program, also compute a trace of the execution
- Use the trace to propagate the selection information
  - Slicing with respect to a given execution means we are doing *dynamic program slicing*
  - Operators for propagating selection information have nice round-tripping properties

Intuition on Dependency / Galois Connections

- Fix a pair of sets, X of inputs, and Y of outputs, and fix an input and output $x \in X, y \in Y$
- We write $D(x, y)$ to mean *x depends on y*
- For $Y' \subset Y$ we write $\triangleleft_D(Y')$ to mean the set of inputs that are *consumed by* Y'
- For $X' \subset X$ write $\triangleright_D(X')$ to mean the set of outputs *produced* by X'
- We will write $\triangleleft_D \dashv \triangleright_D : \mathbb{P}(X) \to \mathbb{P}(Y)$ to mean that the relation forms a Galois connection
-
- Can consider $\mathbb{P}(X), \mathbb{P}(Y)$ as Boolean algebras, meaning we can derive De Morgan duals:
    - For a function between boolean algebras $f : \mathcal{A} \to \mathcal{B}$ we write $f^\circ : \mathcal{A} \to \mathcal{B} = \neg_\mathcal{B} \circ f \circ \neg_\mathcal{A}$
    - A Galois connection has a De Morgan dual, composed of the De Morgan duals of it's 2 components

Linked Outputs

- In POPL'22, exploited the De Morgan dual construction to link outputs which share input data
- Intuitively:
  - Backward slice against selected output in V1 to find data needed to compute it
  - Negate the result of backward slicing
  - Forward slice against the negated selection into V2
  - Negate the result of forward slicing in V2
- Resulting selection is all output data in V2 which shares some input with the original selection



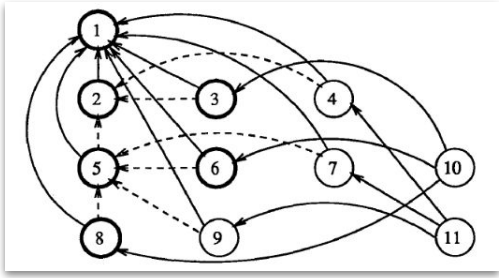
year	country	energyType	output
2015	China	Bio	10.3
2015	China	Hydro	96
2015	China	Solar	44
2015	China	Wind	145

BUT! Performance…

- Despite nice properties of Galois slicing and the relative simplicity of traces, approach is limited
- Repeated slicing queries are very costly performance wise, as need to rewind and replay the entire computation anytime you want to slice against I/O
- Also, counter-intuitive:
  - Aim of slicing is to identify independent strands of execution
  - Doesn’t quite make sense to replay the entire program when you only care about a certain part
- How can we overcome this limitation?

Dynamic Dependence Graphs

- Introduced in 1990 [2]
- Edges represent data dependencies, and separately control dependencies
  - In original work, control dependency reflects nesting of statements
- Naturally represents independent parts of a given computation
- Historically used in the setting of dynamic slicing for imperative languages
  - Much less common for higher-order functional languages
  - Never used in conjunction with Galois connections [3]



(2) Hiralal Agrawal and Joseph R. Horgan. 1990. Dynamic program slicing. SIGPLAN Not. 25, 6 (Jun. 1990), 246–256. https://doi.org/10.1145/93548.93576
(3) Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2006. Adaptive functional programming. ACM Trans. Program. Lang. Syst. 28, 6 (November 2006), 990–1034. https://doi.org/10.1145/1186632.1186634

Galois Slicing Using Dependence Graphs?

	Galois Connections	Dynamic Dependence Graphs
Pros	• Nice round-tripping properties • De Morgan duals can characterize related inputs/outputs • Compositional	• Directly represent dependency structure • Slicing queries can be fast • Conceptually simple
Cons	• Repeated queries are slow • Ties the entire system to a given language	• Tied to imperative languages as a model • Not designed for higher-order programming constructs

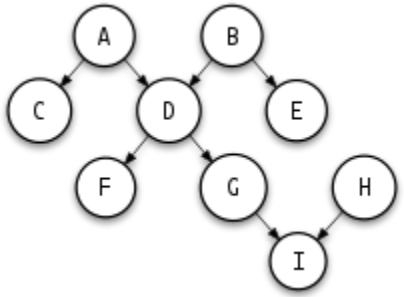
Can we combine the nice mathematical properties of Galois slicing with performance of dynamic dependence graphs?

Can we combine the elegance of Galois connections, with performance and behaviour of dynamic dependence graphs?

Progress to Date

Towards Graphical Galois Slicing

```
let filter' = [[0, 0, 0],
               [3, 7, 1],
               [0, 0, 0]];
    filter  = [| nth2 i j filter' | (i, j) in (3, 3) |];
    image   = [| nth2 i j data | (i, j) in (5, 5) |]
in convolve image filter wrap
```



Basic approach: decompose bidirectional approach to 2 steps:

1. Evaluate program to dependency graph
2. Perform slicing queries against nodes in the graph

image

15	13	6	9	16
12	5	15	4	13
14	9	20	8	1
4	10	3	7	19
3	11	15	2	9

filter

-2	-1	0
-1	1	1
0	1	2

output

1	2	-1	1	5
-1	1	2	-1	1
0	0	1	0	1
0	1	-2	0	1
0	3	0	2	2

Led to initial successes, but raised important questions, which we focus on in the rest of the talk

Backwards Slicing as Reachability

- In graphical setting, backwards slicing from some output selection is equivalent to computing reachable set
  - Much simpler notion of selection propagation than the old method
- Algorithms for computing reachability are linear in the size of the reachable set, meaning backward slicing very efficient:

Test-Name	Trace-Eval	Trace-Bwd	Graph-Eval	Graph-Bwd
convolution/edgeDetect	812.6	580.9	1490.3	5.6
convolution/emboss	656.6	438.1	1107.8	3.5
convolution/gaussian	732.1	453.9	1187.8	4.6
graphics/grouped-bar-chart	96.6	71.8	129.6	3.2
graphics/line-chart	131.1	95.9	200.5	2.2
graphics/stacked-bar-chart	55.9	43.3	70.2	2.0
slicing/dtw/compute-dtw	146.2	131.6	254.3	1.8

Overhead of building graph goes away when slicing multiple times

Easy (and Inefficient) Forward Slicing

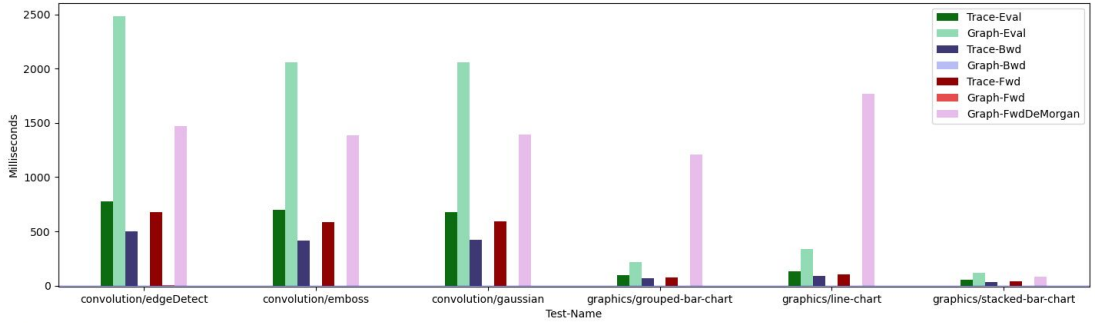
- De Morgan dual allows for naive implementation of forwards slicing:
  - Invert selection (set complement within outputs)
  - Invert all edges in graph
  - “Backward slice” against the inverted selection in the inverted graph
  - Invert the resulting input selection
- No need to define a separate forward slicing procedure
- However, worst case takes time proportional to entire graph + extra (for set operations)

More Efficient Forward Slicing

- Instead, we define a more efficient procedure:
  - Enqueue pending nodes
  - For each pending node, check if all out-neighbours are in current slice, if yes add pending node to slice, and add it’s in-neighbours to pending set, if no, proceed to next pending node
  - Repeat until we cannot remove any pending nodes from the queue, then return the current slice

Performance Considerations

- Instead of slicing taking time proportion to original execution, slicing now takes time proportional to the size of the slice. Means slicing several orders of magnitude faster
- Constructing the graph more expensive than constructing the trace, in systems where large numbers of slicing queries are being run this issue resolved




```
(base) oz20977@IT080942:~/Documents/fluid-proj/fluid$ python plot_bench.py
                          Trace-Eval  Trace-Bwd  Trace-Fwd  Graph-Eval  Graph-Bwd  Graph-Fwd  Graph-FwdDeMorgan
Test-Name
convolution/edgeDetect        775.69     497.75     680.80     2481.52       0.47       3.52            1471.33
convolution/emboss            698.80     417.37     586.77     2058.83       0.25       1.22            1388.11
convolution/gaussian          676.37     422.51     592.98     2059.10       0.26       1.29            1393.25
graphics/grouped-bar-chart     96.93      65.83      77.07      219.54       0.35       0.09            1209.06
graphics/line-chart           130.67      89.94     106.73      339.16       0.54       0.20            1765.32
graphics/stacked-bar-chart     53.93      34.71      40.12      121.52       0.23       0.03              80.32
```

Some Notation

- For fixed $G = (V, E)$, we consider $G^{-1} = (V, E^{-1})$, where E^{-1} is E with the direction of edges reversed
- We consider **;** to stand for graph composition or union
- Fix a relation $D \subset X \times Y$, then define the following functions:
  - $\triangleleft_D(Y') := \{x \in X | \exists y \in Y'. D(y, x)\}$
  - $\triangleright_D(X') := \{y \in Y | \forall x \in X'^c. \neg D(y, x)\}$

Some Notation

- For fixed $G = (V, E)$, we consider $G^{-1} = (V, E^{-1})$, where E^{-1} is E with the direction of edges reversed
- We consider **;** to stand for graph composition or union
- Fix a relation $D \subset X \times Y$, then define the following functions:
  - $\triangleleft_D(Y') := \{x \in X | \exists y \in Y'. D(y, x)\}$
  - $\triangleright_D(X') := \{y \in Y | \forall x \in X'^c. \neg D(y, x)\}$

Recasting Linked Outputs on the Graph

- In POPL, composed backward slicing with the De Morgan dual of forward
- In graph setting, $(fwd(G))^{\circ} = bwd(G^{op})$
- Lets us compute linked outputs for computations G, H just using backward:
    - $bwd(G) \circ bwd(H^{op}) = bwd(G; H^{op})$
    - (;) is more than just graph union
    - We think of graphs having types representing I/O
    - Then $(\cdot)^{op}$ is contravariant

$bwd(G^{op})$is inverse of reachability, then linked outputs is asking the question “Who can reach anything I can reach?”

Linked Visualizations (cont.)

- More generally, consider a pair of computations as a computation with a product output
  - Now, linked outputs is just $bind(G, G^{op})$
- The above invites the question, what does it mean to slice over G^{op}, G ?
- We will give an example of that next, before moving onto the remaining research problem

Linked Inputs

Linked Inputs: Example

Current: Graph Semantics